CollaRAG: Graph-Grounded Division of Labor for Multi-Agent Retrieval

Anonymous Submission

Abstract

Existing Retrieval-Augmented Generation (RAG) systems apply a uniform retrieval pipeline to every query, whether it demands a single-fact lookup, multi-hop reasoning, or cross-document synthesis. This monolithic design over-processes simple queries while under-serving complex ones, because one pipeline cannot match both the cost and the evidence shape of every query type. We introduce COLLARAG, a multi-agent RAG framework built on the principle of *graph-grounded division of labor*: an intent-aware triage routes each query to a specialist agent whose tool permissions, reasoning protocol, and memory scope fit the query's demand profile, and every routing, reasoning, memory, and capability decision is conditioned on a shared knowledge graph. COLLARAG makes four contributions. **First**, *Intent-Aware Query Triage*, a two-level classifier that routes each query to the best-suited specialist via one-shot, irreversible dispatch, replacing the one-pipeline-fits-all bottleneck and enabling clean per-agent ablation. **Second**, *Graph-Grounded Plan-Execute-Reflect* (G-PER), a multi-hop reasoning protocol whose Plan, Execute, and Reflect phases are each anchored to the knowledge graph: the plan is validated against the KG schema before any retrieval budget is spent, execution shares a frontier budget across steps, and reflection checks *structural* evidence completeness and probes the KG for contradictions—checks that graph-agnostic ReAct/Reflexion cannot perform. **Third**, *Graph-Grounded Dual-Layer Memory*, whose shared layer caches evidence subgraphs retrieved by entity-neighborhood overlap (not text match) and whose private layer stores graph plan templates and typed gap lessons that feed G-PER's Plan and Reflect, separating structural learning from cache hits. **Fourth**, *Graph-Conditioned Capability Specialization*, which makes the per-agent tool-permission matrix adaptive to the query's structural footprint on the KG—withdrawing graph-traversal tools and adding text fallback in graph-poor regions—a graph-conditioned capability mechanism absent from prior strategy-level routing. Five agents—a preprocessor and four task specialists—share a pool of 31 retrieval, reasoning, verification, and memory tools, with access governed by a graph-conditioned tool-permission matrix that adapts each agent's toolset to the query's structural footprint on the KG. On six QA benchmarks spanning single-hop (NQ, PopQA), multi-hop (HotpotQA, 2WikiMultiHopQA, MuSiQue), and long-form summarization (ASQA), COLLARAG outperforms nine baselines including the previous SOTA HippoRAG2 [Gutierrez et al., 2025] under both Qwen3-8B and Qwen3-32B backbones. The largest gain is on ASQA, +7.2 F1 (Qwen3-8B), where no baseline runs a dedicated aggregation pipeline; on factoid and multi-hop sets, where baselines already retrieve well, gains come from reranking, sub-query decomposition, and reflection (+1.8 to +5.7 F1, Qwen3-8B). Ablation studies confirm that intent-aware routing is the single most impactful component, contributing 5.9–12.2 EM points when removed.

Keywords: Retrieval-Augmented Generation, Multi-Agent Systems, Query Routing, Multi-hop Reasoning, Division of Labor

1 Introduction

Large language models (LLMs) have achieved remarkable performance across a wide range of language understanding and generation tasks [Wei et al., 2022]. Despite their success, LLMs suffer from three well-documented limitations that hinder reliable deployment in knowledge-intensive applications. First, *knowledge staleness*: the parametric knowledge of an LLM is frozen at its training cutoff, rendering it unable to reflect recent events or updated domain facts. Second, *hallucination*: LLMs can generate

fluent but factually incorrect content, confidently presenting fabricated information indistinguishable from genuine knowledge [Gao et al., 2024]. Third, *provenance opacity*: generated answers lack source attribution, making it impossible for users to verify claims or trace them back to authoritative evidence. These shortcomings are especially problematic in enterprise question answering, legal and medical consultation, and scientific literature review, where factual accuracy and traceability are non-negotiable requirements.

Retrieval-Augmented Generation (RAG) [Lewis et al., 2020] has emerged as a leading paradigm for addressing these limitations. By retrieving relevant passages from an external knowledge base and prepending them to the LLM prompt at inference time, RAG grounds generation in verifiable evidence without retraining the model. This architecture offers three practical advantages: (1) knowledge can be updated by modifying the external corpus rather than retraining billion-parameter models; (2) generated answers can be traced back to specific source passages, enabling auditability; (3) the same LLM can serve multiple domains by swapping the underlying knowledge base. Backed by mature infrastructure—vector databases such as FAISS [Johnson et al., 2019] and Milvus, embedding models [Chen et al., 2024], and orchestration frameworks like LangChain and LlamaIndex—RAG has become one of the most widely adopted paradigms for deploying LLMs in knowledge-intensive production systems.

The standard RAG pipeline follows a sequential four-stage architecture: *chunking* (splitting documents into passages), *indexing* (encoding chunks into dense vectors), *retrieval* (finding top-k relevant chunks via similarity search), and *generation* (feeding retrieved chunks to the LLM for answer synthesis). Existing RAG methods have advanced the retrieval substrate along three directions: *text-based* methods enhance passage retrieval through hierarchical indexing (e.g., RAPTOR [Sarthi et al., 2024]) and hybrid dense-sparse fusion; *graph-based* methods such as TOG [Sun et al., 2024] and G-Retriever [He et al., 2024] exploit entity-relation structures for multi-hop reasoning; and *hybrid* methods—GraphRAG [Edge et al., 2024], LightRAG [Guo et al., 2024], HippoRAG [Gutierrez et al., 2024]—combine textual evidence with graph traversal, achieving current state-of-the-art results on multi-hop QA benchmarks.

Yet across all three families, a fundamental design assumption remains unchallenged: *every query passes through the same retrieval pipeline*. Whether a user asks a trivial factual question or a complex comparative analysis requiring evidence chains across multiple documents, the system invokes identical retrieval logic, tool budgets, and reasoning strategies. We term this pervasive pattern *retrieval monolithism*.

Retrieval monolithism creates three concrete problems:

- **Simple query over-processing.** Single-hop factual questions that need only one passage lookup are funneled through sophisticated multi-step pipelines (e.g., GraphRAG's community summarization or HippoRAG's PageRank traversal), incurring unnecessary latency and token cost without quality improvement. In mixed query workloads that include factoid, multi-hop, and summarization requests, a substantial fraction of queries are single-hop, making uniform heavy pipelines inefficient.
- **Complex query under-reasoning.** Multi-hop questions require chaining evidence across multiple documents, often involving entity comparison, temporal reasoning, or causal inference. Existing methods apply the same retrieval budget regardless of query complexity. Even graph-based systems typically perform retrieval in a single forward pass without iterative evidence verification: when the initial retrieval misses a critical linking entity, most systems lack a mechanism to detect and repair the gap.
- **Summarization query under-aggregation.** Queries requesting overviews or syntheses require global information aggregation across many passages, not point retrieval of specific facts. Existing multi-hop QA-oriented RAG systems rarely treat summarization as a first-class routed intent with a dedicated aggregation pipeline. As a result, summarization requests are often handled by the same top-k retrieval mechanism used for factoid QA, which may cover only a small portion of the relevant corpus and produce incomplete summaries.

These limitations are structural consequences of the monolithic design, not bugs in individual systems.

Addressing them requires rethinking the retrieval architecture itself.

We draw inspiration from the *division of labor* principle. In organizational design, a hospital triages patients by symptom severity, routing a headache to general medicine and a fracture to orthopedics. Each department has specialized equipment and protocols optimized for its patient population. No hospital sends every patient through the same diagnostic pipeline. Similarly, an intelligent retrieval system should route queries to specialized agents based on query characteristics, equipping each agent with the tools and reasoning strategies best suited to its query type.

We introduce COLLARAG, a framework that replaces the monolithic retrieval pipeline with intent-conditioned specialist execution. COLLARAG routes queries into five handling modes—greeting short-circuit, SingleHop, MultiHop, Summarization, and Other—supported by a Preprocessor agent and four task-specialist agents. An *intent-aware query triage* module classifies incoming queries and dispatches each to the matching handler. Each task agent draws from a shared pool of 31 retrieval, reasoning, verification, and memory tools, with access governed by an adaptive, *graph-conditioned tool-permission matrix* that specializes each agent's toolset to the query's structural footprint on the knowledge graph (degrading to a static matrix when the graph is unavailable). The MultiHop agent employs a novel *G-PER* (graph-grounded Plan-Execute-Reflect) workflow. A *dual-layer memory system* enables cross-agent coordination and per-agent progressive learning.

Specifically, COLLARAG makes four technical contributions:

1. **Intent-Aware Query Triage** (§3.1). We propose a two-level intent classifier—a fast regex-based greeting filter cascading into LLM semantic classification with keyword-based rule fallback—that categorizes queries into five handling modes (`greeting`, `single_hop`, `multi_hop`, `summarization`, `other`) and dispatches each to the matching handler via one-shot deterministic routing. Unlike prior adaptive retrieval methods that mainly switch among retrieval depths or strategy templates, our triage operates at the agent level: different intents are handled by specialists with distinct tool permissions, reasoning protocols, synthesis procedures, and private memories.

2. **Graph-Grounded Plan-Execute-Reflect (G-PER)** (§3.2). For complex multi-hop queries, we re-ground the three reasoning phases in the knowledge graph rather than treating it as one optional tool. *Plan* emits graph operations (link/traverse/attribute/aggregate) validated against the KG schema, so malformed steps are caught before any retrieval budget is spent; *Execute* runs a frontier of pending steps under a shared global budget, letting an easy step donate unused iterations to a harder downstream one; *Reflect* replaces the scalar quality gate with a structural-completeness check that emits typed evidence gaps (`missing_attribute`/`missing_path`/`missing_relation`) driving targeted repair, plus an adversarial probe that searches the KG for contradicting relations. Each mechanism degrades gracefully: in a graph-poor setting G-PER collapses to text-only PER, which is itself the ablation that isolates the contribution of graph grounding. This addresses three weaknesses of flat ReAct loops that prior plan/reflect work [Wang et al., 2023, Shinn et al., 2023] shares: absence of schema-grounded planning, fixed per-step budgets, and scalar (non-structural) gap detection.

3. **Graph-Grounded Dual-Layer Memory** (§3.3). The *shared layer* caches evidence subgraphs retrieved by entity-neighborhood overlap rather than text-hash match, so reuse reflects structural recurrence of the same entity neighborhood—legitimate on non-repeating benchmarks—and is separated from the legacy QA cache. The *private layer* stores graph plan templates (keyed by compositionality + entity signature, recalled as a plan prior) and typed gap lessons (G-PER's Reflect output, fetched pre-emptively before future reasoning), composing Contribution 3 with Contribution 2; a text-only lesson channel—the tool-sequence failures G-PER's Plan recalls as avoidance guidance—serves as the graph-poor counterpart, keeping the fallback functional rather than vestigial. This separates structural learning from cache hits, which a purely textual memory cannot.

4. **Graph-Conditioned Capability Specialization** (§3.4). We make the per-agent tool-permission

matrix adaptive to the query's structural footprint on the KG: a cheap, LLM-free probe (entity hit rate, path existence, density) withdraws graph-traversal tools and adds text fallback in graph-poor regions, and collapses a specialist's KG projection (attribute / path / community) when the graph cannot support it. This turns a static capability gate into an adaptive, graph-conditioned one, and supplies the concrete mechanism—plus a measurable ablation—for the agent-level versus strategy-level routing distinction that prior adaptive-retrieval work [Jeong et al., 2024] only asserts.

We evaluate COLLARAG on six QA benchmarks that collectively span three intent types: single-hop factoid QA (NQ [Kwiatkowski et al., 2019], PopQA [Mallen et al., 2023]), multi-hop reasoning (HotpotQA [Yang et al., 2018], 2WikiMultiHopQA [Ho et al., 2020], MuSiQue [Trivedi et al., 2022]), and long-form summarization (ASQA [Stelmakh et al., 2022]). This mixed-intent evaluation directly validates the core hypothesis: a system designed for division of labor should excel when queries demand different retrieval strategies. Using both Qwen3-8B and Qwen3-32B as backbone LLMs, COLLARAG achieves state-of-the-art results across all benchmarks and backbone scales, outperforming the previous best method HippoRAG2 [Gutierrez et al., 2025] on every dataset, with gains of 5.7 F1 on 2WikiMultiHopQA (Qwen3-8B) and 5.1 F1 on HotpotQA (Qwen3-32B), and the largest gain of 7.2 F1 on ASQA (Qwen3-8B) where the dedicated Summarization agent has no baseline counterpart. Comprehensive ablation studies confirm that intent-aware routing is the single most impactful component (removing it causes -11.7 EM on 2WikiMultiHopQA), validating the division-of-labor hypothesis.

2 Related Work

2.1 RAG Foundations and Retrieval Enhancement

RAG [Lewis et al., 2020] established the retrieve-then-generate paradigm, pairing parametric LLMs with non-parametric passage retrieval to ground generation in external evidence. Many vanilla RAG implementations rely on DPR-style bi-encoder retrieval [Karpukhin et al., 2020] for open-domain QA, and Fusion-in-Decoder [Izacard and Grave, 2021] scaled this approach to processing many retrieved passages in parallel. Subsequent work has refined the retrieval stage through multiple dimensions: hybrid retrieval combining dense and sparse (BM25 [Robertson and Zaragoza, 2009]) signals via reciprocal rank fusion [Cormack et al., 2009]; cross-encoder reranking [Nogueira and Cho, 2019] for more precise relevance estimation; and query rewriting [Ma et al., 2023] for bridging the vocabulary gap between queries and documents. RAPTOR [Sarthi et al., 2024] organizes retrieval via recursive abstractive trees, enabling retrieval at multiple abstraction levels. These methods enhance retrieval quality but apply a uniform pipeline to all queries regardless of complexity—the monolithic limitation that COLLARAG addresses.

2.2 Graph-based and Hybrid RAG

Knowledge graphs provide structured pathways for multi-hop reasoning. GraphRAG [Edge et al., 2024] performs community detection on entity graphs using the Louvain algorithm [Blondel et al., 2008] and generates hierarchical summaries for query-focused retrieval. LightRAG [Guo et al., 2024] introduces dual-level (key-entity and high-level topic) graph retrieval for improved efficiency. TOG [Sun et al., 2024] uses LLMs to iteratively traverse knowledge graph triples, while G-Retriever [He et al., 2024] integrates graph neural networks with LLMs for subgraph retrieval. HippoRAG [Gutierrez et al., 2024] draws on hippocampal indexing theory, implementing personalized PageRank over a knowledge graph to mimic human associative memory, and HippoRAG2 [Gutierrez et al., 2025] extends this with deeper memory integration and passage-level retrieval. These systems optimize *what* is retrieved by enriching the knowledge substrate, but do not adapt *how* retrieval is orchestrated based on query type—an orthogonal dimension that COLLARAG addresses through intent-aware routing. COLLARAG is complementary to

these retrieval substrates: graph traversal, dense retrieval, and hybrid retrieval can all serve as tools inside specialist agents.

2.3 Adaptive RAG and Query Routing

Recent work has begun to challenge the one-size-fits-all assumption. Self-RAG [Asai et al., 2024] trains an LLM to decide *when* to retrieve and to self-critique generated outputs using reflection tokens, but applies the same retrieval mechanism once triggered. Adaptive-RAG [Jeong et al., 2024] trains a query-complexity classifier (on T5-Large) to route among three strategies: no retrieval, single-step retrieval, and multi-step retrieval. CRAG [Yan et al., 2024] evaluates retrieval quality and triggers corrective web searches when confidence is low. These approaches adapt retrieval decisions at the strategy level—whether to retrieve, how many retrieval steps to take, or whether to correct retrieval results—but route to pre-defined strategies within a single pipeline. COLLARAG performs routing at the agent level: the selected handler differs not only in retrieval depth but also in tool permissions, reasoning protocol (single-round vs. G-PER vs. Map-Reduce), memory scope, and synthesis procedure, realizing a true division of labor rather than a strategy-selection switch.

2.4 Multi-hop Reasoning and Query Decomposition

Multi-hop QA requires chaining evidence across multiple passages. Chain-of-thought prompting [Wei et al., 2022] enables LLMs to articulate intermediate reasoning steps. IRCoT [Trivedi et al., 2023] interleaves retrieval with chain-of-thought steps, using each reasoning step to guide the next retrieval query, achieving strong results on HotpotQA, 2Wiki, and MuSiQue. Self-Ask [Press et al., 2023] and Least-to-Most prompting [Zhou et al., 2023] decompose complex questions into simpler sub-questions that can be answered sequentially. Plan-and-Solve [Wang et al., 2023] generates explicit plans before execution. These methods improve multi-hop reasoning within a single pipeline. COLLARAG's MultiHop agent builds on these ideas but introduces a key structural innovation: each of the plan, execute, and reflect phases is *grounded in the knowledge graph*—the plan is schema-validated, execution shares a frontier budget, and reflection checks structural evidence completeness and probes for graph-level contradictions—so the reasoning protocol exploits graph structure that text-only plan/reflect methods [Wang et al., 2023, Shinn et al., 2023] cannot access.

2.5 Agentic RAG and Multi-Agent LLM Systems

The rise of LLM-based agents [Xi et al., 2023] has introduced tool use into RAG. ReAct [Yao et al., 2023] interleaves reasoning and action (tool invocation) in a single loop. Toolformer [Schick et al., 2023] teaches LLMs to decide when and how to call external tools. Multi-agent frameworks such as MetaGPT [Hong et al., 2024] and AutoGen [Wu et al., 2024] enable multiple agents to collaborate through conversational protocols. However, these multi-agent systems assume agents need to *negotiate*—exchanging messages, debating solutions, or voting on outcomes. COLLARAG adopts a fundamentally different *dispatch* model: a classifier makes a one-shot routing decision, and the assigned specialist agent executes independently without inter-agent dialogue. This design prioritizes deterministic reproducibility and clean ablation boundaries over emergent collaboration, while still achieving the benefits of specialization through the tool-permission matrix.

2.6 Memory-Augmented RAG

Standard RAG systems are stateless: each query is processed independently with no memory of past interactions. HippoRAG [Gutierrez et al., 2024] introduced biologically-inspired long-term memory into RAG through knowledge graph-based indexing. MemoRAG [Qian et al., 2025] builds a global memory model that recalls query-specific clues to guide retrieval over long contexts. ExpeL [Zhao

et al., 2024a] demonstrates that LLM agents can learn from accumulated experience, extracting insights from past task trajectories to improve future performance. Reflexion [Shinn et al., 2023] stores verbal self-reflections as episodic memory to prevent repeating mistakes. COLLARAG contributes a *dual-layer* memory architecture that separates cross-agent shared memory (QA caching, routing statistics, entity aliases) from per-agent private experiential memory (tool sequences, quality scores, lessons learned). This separation enables both collaborative knowledge sharing and isolated experiential learning, with clean ablation boundaries between the two layers.

2.7 Retrieval-Augmented Summarization

While most RAG systems focus on factoid QA, real-world queries often demand summarization or synthesis across many passages. The Map-Reduce paradigm for distributed text processing [Dean and Ghemawat, 2008] has been adapted to LLM-based summarization: each passage is independently summarized (Map), then all summaries are aggregated into a coherent overview (Reduce). GraphRAG [Edge et al., 2024] uses community-level summaries for global queries but does not offer a dedicated summarization agent. Existing RAG surveys [Gao et al., 2024, Zhao et al., 2024b] identify summarization as an underserved query type in current systems. COLLARAG addresses this gap with a dedicated Summarization agent that implements a full Map-Reduce pipeline: broad retrieval ($k = 50$), per-chunk LLM summarization, and final aggregation. This makes summarization a first-class routed intent within the same multi-agent RAG framework, rather than an external summarization chain or a generic retrieval mode.

3 Method

COLLARAG is built on a single thesis: a retrieval system should not push every query through the same pipeline, because queries differ in the amount of evidence, the shape of reasoning, and the kind of aggregation they require. We realize this thesis as an orchestrator that, for each query, decides *which* specialist should handle it, equips that specialist with a bounded set of tools, and lets it follow a protocol matched to the query type. Figure 1 sketches the end-to-end flow. Concretely, a query passes through six stages: (i) intent classification, (ii) a greeting short-circuit, (iii) a QA-cache lookup, (iv) query preprocessing, (v) dispatch to one of four task-specialist agents, and (vi) post-processing that writes back to memory. The four contributions of this paper sit at the decision points of this flow: how a query is routed (§3.1), how the hardest queries are reasoned about (§3.2), how experience accumulates across queries (§3.3), and how each specialist's capability is conditioned on the graph (§3.4).

3.1 Intent-Aware Query Triage

The triage module answers one question per query: which specialist should run? It is a two-level cascade rather than a single classifier, because greetings are trivially detectable and should never pay the cost of a semantic LLM call. Level 1 is a regex filter for greeting patterns; a hit returns a direct LLM response and skips the rest of the pipeline. Level 2 sends the non-greeting query to the backbone LLM, which returns one of four labels—`single_hop`, `multi_hop`, `summarization`, or `other`—and a keyword rule fallback covers parse failures (summarization and comparative keywords, else `single_hop`). The full classification prompt is given in Appendix A.1. Formally, the routing decision is

$$a^*(q) = \text{dispatch}\big(\underbrace{L_2(q)}_{\text{LLM label}} \circ \underbrace{L_1(q)}_{\text{regex}}\big), \quad a^* \in \{\text{SH}, \text{MH}, \text{SM}, \text{OT}\} \cup \{\text{greet}\}, \tag{1}$$

where L_1 short-circuits greetings and L_2 labels the rest. Routing is one-shot and irreversible: once a^* is chosen, the query stays with that agent. We impose this on purpose, so that each specialist's marginal contribution can be measured cleanly (§5) and so that a single misclassification cannot cascade through re-routing.

TODO: Architecture of COLLARAG

Style: icon-driven, minimal text, Comic Sans MS. Do NOT label the four contributions as C1–C4; let the components embody them. All boxes outlined/transparent so connection lines stay visible. Four stacked rows, centered. *Row 1* (pipeline, left-to-right): a speech-bubble query → a branching fork with a lock (one-shot dispatch) → four specialist icons in a row — SingleHop (pin), MultiHop (a magnifier over graph-nodes; inside it three tiny phase icons clipboard→gears→refresh for Plan/Execute/Reflect), Summarization (stack), Other (globe) → a checkmark answer. *Row 2*: a shared toolbox with a valve/gate between it and the agent row (graph-conditioned tool permission; a graph-poor branch greys out traversal tools and lights up a web-fallback icon). *Row 3*: a Knowledge-Graph panel (nodes-and-edges), centered, *the same size as the memory block*, with dashed arrows rising into the fork, the MultiHop magnifier, and the valve. *Row 4* (bottom): the memory block — a database with two overlapping entity circles (shared evidence-subgraph overlap cache) next to a stacked drawer (private plan templates + gap lessons), with one arrow from MultiHop's Reflect phase down into the drawer.

Figure 1: Architecture of COLLARAG. A query is triaged and one-shot dispatched to a specialist agent; the MultiHop agent reasons via G-PER (Plan/Execute/Reflect); each agent's toolset is gated by the query's graph footprint; a graph-grounded dual-layer memory underlies all agents and is fed by G-PER's Reflect. (Figure to be drawn.)

Four task specialists. Each label selects an agent whose protocol fits the query's demand profile. SingleHop runs one round of hybrid (dense + BM25) retrieval, fuses the two rank lists by reciprocal rank fusion,

$$s(d) = \sum_{i \in \{\text{vec},\text{bm25}\}} \frac{1}{\text{rank}_i(d) + k}, \tag{2}$$

optionally reranks, and generates after a one-shot quality check—the cheapest path, sufficient for a single-fact lookup. MultiHop runs the G-PER workflow of §3.2. Summarization runs a Map-Reduce pipeline that retrieves a broad pool ($k = 50$), summarizes each passage locally, and reduces the summaries into one overview, a shape of aggregation no factoid-retrieval pipeline provides (Appendix A.5). Other falls back to web search with optional local retrieval for out-of-domain queries. A Preprocessor agent rewrites the query (expansion, coreference resolution, follow-up merging) before triage; it holds no retrieval tools and only cleans the query text (prompt templates in Appendix A.2).

Tool-permission matrix. All five agents share one pool of 31 tools, but each agent sees only a curated subset (Table 1). The matrix is the infrastructure-level enforcement of specialization: MultiHop alone can call graph path search, sub-query decomposition, and claim verification, while SingleHop works with a focused 15-tool set. This prevents a nominally specialized agent from quietly behaving like a generalist. The complete tool inventory with per-tool descriptions is listed in Appendix B.

TODO: Two-Level Query Triage

A decision tree. Root: query q. Level 1: regex greeting filter $\rightarrow$ on hit, short-circuit to a direct LLM answer. Level 2 (non-greeting): an LLM semantic classifier with a dashed fallback edge for parse failures, branching into four leaves: `single_hop`→SingleHop, `multi_hop`→MultiHop, `summarization`→Summarization, `other`→Other. Annotate the one-shot, irreversible dispatch with a lock icon on the leaf edges.

Figure 2: Two-level query triage. L_1 short-circuits greetings; L_2 labels the rest into four specialist routes. (Figure to be drawn.)

3.2 Graph-Grounded Plan-Execute-Reflect (G-PER)

Multi-hop queries are the hardest case: the answer depends on a chain of facts spread across passages, and a single retrieval pass usually returns one link without the others. A flat ReAct loop [Yao et al., 2023] interleaves thought and action greedily, which (i) never commits to an explicit plan, (ii) has no principled way to notice that evidence is incomplete, and (iii) is *graph-agnostic*—it treats the knowledge graph as one optional tool among many, so the structure that multi-hop evidence forms is invisible to the reasoning protocol. We re-ground each phase in the knowledge graph and call the result *G-PER*: Plan emits graph operations validated against the KG schema, Execute allocates a shared budget across a frontier of pending steps, and Reflect checks *structural* evidence completeness and probes the KG for contradictions. Crucially, every graph-grounded mechanism degrades gracefully: when the graph store is absent or sparse (the *graph-poor* setting), G-PER collapses to text-only PER, which is itself the ablation that isolates the contribution of graph grounding (§5.2). The full prompt templates for each phase are in Appendix A.4.

Plan (schema-constrained). Given q, the LLM produces a retrieval plan as a bounded list of *graph operations*, not free-text sub-queries:

$$P = \big[(op_i, e_i, r_i, t_i)\big]_{i=1}^{n}, \qquad op_i \in \{\textsf{link}, \textsf{traverse}, \textsf{attribute}, \textsf{aggregate}\}, \quad n \le n_{\max}{=}5, \tag{3}$$

where e_i is an entity, r_i a relation type, and t_i a target entity. Before any retrieval budget is spent, each step is validated against the KG schema:

$$V(s_i) = \big(r_i \in \mathcal{S} \cup \{\perp\}\big) \wedge \big(e_i \in \mathcal{N} \ \vee \ op_i{=}\textsf{aggregate}\big), \tag{4}$$

where $\mathcal{S}$ is the set of predicates observed in the graph (the only ontology available) and $\mathcal{N}$ the node set. A step failing V is *degraded* to a free sub-query $\{q_i, \emptyset\}$ rather than dropped, so a malformed plan never blocks the pipeline—the agent still answers, just without that step's graph structure. If the whole plan is unparseable, it degrades to $[\{q, \emptyset\}]$. Static validation is the first graph-grounded check that text-only Plan-and-Solve [Wang et al., 2023] cannot perform: a free-text plan has no schema to check against.

Tool Category	PP	SH	MH	SM	OT
Retrieval (6)	0	4	6	4	1
Entity (3)	0	2	3	1	1
Reasoning (3)	0	2	3	1	0
Preprocessing (3)	3	0	0	0	0
Memory (3 read)	2	2	3	2	2
Web (2)	0	0	0	0	2
IO (2)	0	2	2	2	2
Verification (3)	0	1	3	1	0
Graph Analysis (2)	0	0	2	0	0
Evidence (4)	0	2	4	3	1
Total	5	15	26	14	9

Table 1: Tool-permission matrix. PP=Preprocessor, SH=SingleHop, MH=MultiHop, SM=Summarization, OT=Other. The MultiHop agent has the richest toolkit (26/31), reflecting its need for graph traversal, sub-query decomposition, answer verification, and evidence assessment.

Execute (frontier-based). Steps run as mini-ReAct loops [Yao et al., 2023] that may call tools and see earlier steps' results, but under a *shared global budget* J rather than a fixed per-step cap:

$$r_i = \mathrm{MiniReAct}\big(s_i,\, q,\, \{r_1, \ldots, r_{i-1}\},\, \mathcal{T}_{\mathrm{MH}}\big), \quad \text{using } j_i \le \min(j_{\max},\, J_{\mathrm{rem}}) \text{ iterations}, \tag{5}$$

where $J_{\mathrm{rem}} = J - \sum_{k<i} j_k$ is the remaining frontier budget and $J{=}n_{\max}{\cdot}j_{\max}$ by default. A step that emits a Step Result after $j_i < j_{\max}$ iterations donates the saved $j_{\max} - j_i$ iterations to downstream steps—the adaptive allocation a fixed per-step $j_{\max}$ cannot provide. The graph supplies the natural expansion primitive (expand neighbors / expand path), so the frontier is over evidence paths, not arbitrary text states.

Reflect (structural completeness + adversarial probe). We replace the scalar quality gate with a structural check. A cheap rule-based module infers the query's *required evidence shape* $\sigma(q) \in \{\mathsf{comparison}, \mathsf{bridge}, \mathsf{attribute}\}$ from phrasing and linked entities, then diffs it against the gathered evidence subgraph E:

$$G = \Gamma\big(\sigma(q),\, E\big), \qquad G = \{g_k\}, \quad g_k \in \{\mathsf{missing_attribute}, \mathsf{missing_path}, \mathsf{missing_relation}\}, \tag{6}$$

where each g_k is a *typed, localized* gap (e.g. "no evidence path connecting a to b") carrying a targeted repair query. If $G \neq \emptyset$ and the reflection budget $k < k_{\max}{=}2$ remains, one supplementary mini-ReAct round is fired *per gap*, aimed at the specific missing edge—not the vague "search more" of scalar-gate reflection [Shinn et al., 2023]. When $G = \emptyset$ but the legacy scalar score $\hat{q} < \tau$ (default $\tau{=}0.6$), one text-style reflection round runs as a safety net for non-graph queries. Finally, an *adversarial probe* searches the KG for relations that would contradict the draft answer:

$$\mathrm{verdict} = A\big(q,\, \mathrm{draft},\, \mathrm{KG}\big) \in \{\mathsf{confirmed}, \mathsf{contested}, \mathsf{unsupported}\}, \tag{7}$$

and a contested draft is re-synthesized with the contestations visible to the synthesizer. The final answer is

$$\mathrm{answer} = \mathrm{LLM}_{\mathrm{synth}}\big(q,\, \{r_1, \ldots, r_n\} \cup \{\mathrm{refl}_1, \ldots\} \cup \{\mathrm{adv}\}\big). \tag{8}$$

Separating Plan, Execute, and Reflect is not architectural tidiness alone: it makes each graph-grounded concern independently auditable—schema-validation rate, frontier budget utilization, per-gap repair success, and adversarial-contestation rate—which we exploit in the ablation, where G-PER degrades first to text-only PER and then to flat ReAct.

TODO: G-PER Workflow

Style: icon-driven, Comic Sans MS, transparent outlined boxes. Three graph-grounded phases for the MultiHop agent. *Plan*: LLM $\rightarrow$ graph-op step list $[(op_i, e_i, r_i, t_i)]$, $n \leq 5$, each step validated against the KG schema $V(\cdot)$; invalid steps degrade to free sub-queries (dashed edge). *Execute*: a frontier of pending steps with a *shared* global budget J; a step that finishes early donates its unused iterations downstream. *Reflect & Synthesize*: a structural-completeness checker Γ diffs the required evidence shape against the gathered subgraph and emits typed gaps (`missing_attribute`/`missing_path`/`missing_relation`) that drive *targeted* repair; an adversarial probe A then searches the KG for contradicting relations before final synthesis. Color-code the three phases.

Figure 3: G-PER: graph-grounded Plan-Execute-Reflect. Plan is schema-constrained, Execute is frontier-based, Reflect is structural + adversarial. (Figure to be drawn.)

3.3 Graph-Grounded Dual-Layer Memory

A RAG system that forgets every query after answering wastes the structure that repeats across queries: the same entity neighborhoods, the same successful graph traversal patterns, the same missing edges. COLLARAG keeps two layers of memory with different scopes, and *grounds both in the knowledge graph* so that reuse is structural rather than textual. This directly addresses a failure mode of text-hash memory: on QA benchmarks that do not repeat questions, a text-hash cache hit is suspicious and unexplained, and any "learning" gain is conflated with cache hits. Graph-grounded memory separates the two by reusing *evidence neighborhoods* and *plan shapes*, not question strings.

Shared memory (evidence-subgraph overlap cache). The shared layer keeps a legacy text-hash QA cache plus a *graph-grounded* evidence-subgraph cache. For a high-quality run, COLLARAG stores (E_q, A_q)—the gathered evidence subgraph and answer—keyed by its entity set $\mathcal{E}(q)$. A new query hits the cache when its entities *overlap* a cached entry's neighborhood, not when its text matches:

$$\text{answer}(q) = A_{q'} \;\text{ if }\; \max_{q'} \text{Jaccard}\big(\mathcal{E}(q), \mathcal{E}(q')\big) \geq \theta_o \text{ and } \text{quality}(q') \geq \theta_c, \tag{9}$$

with θ_o=0.5. Overlap reuse is legitimate on non-repeating benchmarks because different questions about the same topic recur over the *same entity neighborhood*—a hit reflects structural reuse, not question duplication, which is the source of the otherwise-puzzling cache-hit rate on standard QA sets. The text-hash cache remains as a fallback when no entities are available.

Agent private memory (graph plan templates + gap lessons). Each agent keeps two graph-grounded stores in per-intent JSONL. *Plan templates* (mechanism 3A) record successful graph plans keyed by compositionality type + entity signature; the MultiHop agent recalls a plan prior before paying for an

TODO: Graph-Grounded Dual-Layer Memory

Style: icon-driven, Comic Sans MS, transparent outlined boxes. Three boxes side by side. *Shared Memory* (graph-grounded): evidence-subgraph overlap cache (keyed by entity sets, retrieved by Jaccard overlap), plus legacy QA cache / routing stats / entity aliases — read by all agents (dashed arrows). *Agent Private Memory* (per-agent×intent): *graph plan templates* keyed by compositionality+entity signature, and *typed gap lessons* (missing_edge) from G-PER's Reflect — isolated per agent; annotate dedup + cold-eviction ($B_{\max}$=200). *Session Memory* (in-process): dialogue history for the Preprocessor. Draw a clear isolation boundary between private buckets; show G-PER's Reflect feeding gap lessons into private memory.

Figure 4: Graph-grounded dual-layer memory. Shared memory reuses evidence neighborhoods (overlap, not text match); private memory stores graph plan templates and typed gap lessons that feed G-PER's Plan and Reflect. (Figure to be drawn.)

LLM plan call (§3.2), so memory improves *structurally* (reusing traversal patterns) rather than by text collision. *Gap lessons* (mechanism 3C) store the typed structural gaps emitted by G-PER's Reflect—`missing_attribute`/`missing_path`/`missing_relation`—as pre-emptive retrieval lessons: a future query whose entities overlap a stored gap fetches the missing edge *before* reasoning begins. This composes Contribution 3 with Contribution 2: G-PER's Reflect output becomes Contribution 3's memory input. A second, text-only channel of the private layer keeps each run's tool sequence and a trace-derived failure lesson ("outcome=lesson"): when a run lands below the quality threshold, the orchestrator distills a structured note—for G-PER, the unresolved typed gaps and reflect count—and *the next G-PER Plan recalls the most-recurring such failures and injects them as avoidance guidance*, so the agent does not repeat a tool-sequence pattern that already produced a low-quality answer. This is the graph-poor counterpart to the structural gap-lessons: when the KG is absent or sparse, the typed gaps are empty and the text-lesson channel is the only learning signal, so the graceful-degradation guarantee of §3.2 extends to memory rather than leaving the text fallback vestigial. Two policies keep the buckets useful: deduplication merges entries with the same signature, incrementing the hit count and refreshing the stored lesson text; and cold-eviction, when a bucket exceeds $B_{\max}$=200, drops the entries that rank lowest on

$$\text{score}_{\text{evict}}(e) = \alpha \cdot \text{hit}(e) + \beta \cdot \text{quality}(e), \tag{10}$$

so rarely-reused, low-quality experiences are recycled first. Private memories remain isolated across agents, which is what makes the per-agent memory ablation meaningful (§5.2).

Session memory. A transient, in-process store keeps per-session dialogue history, which the Preprocessor consumes for coreference and follow-up merging; it is distinct from the two persistent layers and does not survive across sessions.

3.4 Graph-Conditioned Capability Specialization

The static tool-permission matrix (Table 1) enforces specialization at configuration time: MultiHop always receives its 26-tool set regardless of whether the knowledge graph can actually support graph traversal for the current query. This is the weakest link of a purely static design: in a graph-poor region an agent may burn its iteration budget probing an empty graph, while the distinction from prior strategy-level routing [Jeong et al., 2024] remains asserted rather than demonstrated. COLLARAG makes capability specialization *graph-conditioned*: the tool subset an agent receives is a function of the query's structural footprint on the KG.

Graph footprint. Before dispatch, COLLARAG computes a cheap, LLM-free footprint from a capitalized-token entity probe of the query against the graph store:

$$\phi(q) = \big(\,\mathrm{hitRate}(q),\ \mathrm{hasPath}(q),\ \mathrm{density}\,\big), \tag{11}$$

where $\mathrm{hitRate}$ is the fraction of probed entities present in the KG, $\mathrm{hasPath}$ whether a path connects the queried entities, and $\mathrm{density}$ a coarse label of the relevant region.

Dynamic tool permission. The footprint conditions the agent's tool subset (mechanism 4A). For a multi-hop query whose footprint is graph-poor ($\mathrm{hitRate}{=}0$ or sparse-without-path), COLLARAG *withdraws* the graph-traversal tools (`graph_path_search`, `graph_subgraph_extract`, `graph_neighbors`) and *adds* web/dense-hybrid fallback, forcing retrieval by text rather than wasted iterations on an empty graph; for a dense footprint with a confirmed path, traversal is retained. For a single-hop query whose entity is absent from the KG, `graph_neighbors` is dropped (it would return nothing) and dense/BM25 retrieval is relied upon. When the footprint is unavailable (no graph store), the builder falls back to the static matrix—so the graph-poor setting collapses capability specialization to the static baseline, which is itself the ablation that isolates its contribution.

Per-agent KG projection. Each specialist is moreover associated with a KG *projection* (mechanism 4B): SingleHop sees the attribute layer, MultiHop the path layer, Summarization the community layer. The static matrix already realizes these projections (which tools each agent may call); the dynamic rules *collapse* a projection when the graph cannot support it—e.g. a path-view MultiHop query with no path degrades to text retrieval. This turns the tool-permission matrix from a static lookup into an adaptive, graph-conditioned capability gate—a mechanism, absent from prior strategy-level routing [Jeong et al., 2024], for conditioning per-agent capability on the knowledge graph.

4 Experimental Setup

4.1 Datasets

We evaluate on six QA benchmarks spanning three intent categories, using 500 questions per dataset (seed=42, temperature=0):

Single-hop factoid QA. **Natural Questions (NQ)** [Kwiatkowski et al., 2019]: open-domain questions from Google search queries, answerable from a single Wikipedia passage. **PopQA** [Mallen et al., 2023]: entity-centric factoid questions with varying popularity, testing retrieval on long-tail knowledge.

Multi-hop reasoning. **HotpotQA** [Yang et al., 2018]: multi-hop reasoning over Wikipedia, distractor setting. **2WikiMultiHopQA** (2Wiki) [Ho et al., 2020]: questions requiring reasoning across two Wikipedia articles. **MuSiQue** [Trivedi et al., 2022]: compositional 2–4 hop questions resistant to shortcuts.

We report both the answerable subset (MuSiQue-Ans) and the full set including unanswerable questions (MuSiQue-Full).

Long-form summarization. **ASQA** [Stelmakh et al., 2022]: ambiguous factoid questions requiring long-form answers that synthesize information from multiple passages, evaluated with EM and F1.

This mixed-intent benchmark suite is designed to directly test the division-of-labor hypothesis: single-hop benchmarks stress the SingleHop agent, multi-hop benchmarks stress the MultiHop agent with G-PER, and ASQA stresses the Summarization agent with Map-Reduce.

4.2 Baselines

We compare against nine baselines spanning four categories:

LLM-only: Direct prompting without retrieval, run under both the Qwen3-8B and Qwen3-32B backbones.

Text-based: *Vanilla RAG* (chunk + dense retrieval + generation) and *RAPTOR* [Sarthi et al., 2024] (recursive abstractive tree retrieval).

Graph-based: *TOG* [Sun et al., 2024] (LLM-guided graph navigation) and *G-Retriever* [He et al., 2024] (GNN-LLM subgraph retrieval).

Hybrid: *LightRAG* [Guo et al., 2024], *GraphRAG* [Edge et al., 2024], *HippoRAG* [Gutierrez et al., 2024], and *HippoRAG2* [Gutierrez et al., 2025] (the previous state of the art).

4.3 Metrics and Settings

We report three families of metrics.

Answer correctness. *Exact Match (EM)* is 1 if the normalized predicted answer exactly matches the gold answer, else 0. *Token-level F1* is the harmonic mean of token precision and recall against the gold answer after normalization (lowercasing, article/punctuation removal, stemming). Both are standard for open-domain QA [Rajpurkar et al., 2016].

Retrieval quality. *Recall@k* is the fraction of queries whose top-k retrieved passages contain at least one gold supporting evidence. We report Recall@10 on the four multi-hop benchmarks.

Faithfulness. We sample 100 answers per dataset for human + automated evaluation. *Faithfulness* is the share of answer sentences traceable to retrieved evidence (1.0 = every claim is supported). *Hallucination rate* is the share of answer claims with no supporting evidence. *Evidence support rate* is the sentence-level fraction attributable to at least one retrieved passage.

Backbone policy. The main experiment (Table 2) uses both Qwen3-8B and Qwen3-32B. All ablation and analysis experiments (Tables 3–7 and Figures 5–7) use Qwen3-8B. All methods share the same backbone LLM, embedding model, reranker, and evaluation script for fair comparison; all runs use 500 questions per dataset, seed=42, temperature=0.

5 Results

5.1 Main Results

Table 2 presents the main results across six benchmarks spanning three intent types, under both Qwen3-8B and Qwen3-32B backbones.

		Single-hop				*Multi-hop*								*Summ.*	
		NQ		PopQA		2Wiki		HotpotQA		MuSi-A		MuSi-F		ASQA	
Type	**Method**	EM	F1	EM	F1	EM	F1	EM	F1	EM	F1	EM	F1	EM	F1
Qwen3-8B backbone															
LLM-only	Qwen3-8B	.141	.276	.127	.251	.270	.300	.222	.306	.050	.134	.064	.128	.095	.203
Text	Vanilla RAG	.295	.452	.228	.374	.231	.283	.341	.428	.113	.156	.058	.105	.196	.364
	RAPTOR	.344	.508	.288	.441	.312	.330	.352	.428	.132	.156	.085	.153	.254	.412
Graph	TOG	.251	.405	.207	.353	.265	.324	.189	.211	.062	.087	.055	.059	.178	.315
	G-Retriever	.318	.478	.271	.421	.357	.395	.308	.439	.156	.214	.082	.118	.238	.376
Hybrid	LightRAG	.331	.492	.283	.434	.301	.319	.342	.412	.121	.142	.078	.146	.231	.368
	GraphRAG	.357	.521	.312	.463	.314	.327	.371	.498	.169	.295	.137	.158	.261	.401
	HippoRAG	.368	.531	.324	.478	.535	.638	.451	.554	.241	.305	.162	.258	.273	.412
	HippoRAG2	.381	.548	.337	.493	.561	.662	.466	.587	.254	.338	.178	.271	.281	.423
Proposed	CollaRAG (ours)	**.425**	**.595**	**.375**	**.540**	**.602**	**.719**	**.510**	**.628**	**.277**	**.360**	**.189**	**.289**	**.338**	**.495**
Qwen3-32B backbone															
LLM-only	Qwen3-32B	.178	.331	.155	.298	.322	.375	.260	.380	.064	.172	.082	.156	.118	.241
Text	Vanilla RAG	.356	.521	.284	.445	.275	.340	.394	.485	.155	.199	.069	.127	.241	.422
	RAPTOR	.412	.581	.347	.515	.358	.386	.404	.495	.174	.192	.100	.175	.305	.478
Graph	TOG	.312	.475	.261	.421	.318	.382	.240	.260	.097	.115	.070	.077	.223	.378
	G-Retriever	.381	.548	.328	.492	.410	.468	.360	.520	.200	.258	.098	.148	.289	.442
Hybrid	LightRAG	.395	.562	.341	.505	.349	.365	.395	.480	.160	.177	.095	.172	.282	.435
	GraphRAG	.424	.594	.371	.538	.370	.390	.421	.575	.217	.352	.155	.187	.312	.467
	HippoRAG	.435	.605	.384	.553	.590	.710	.500	.630	.285	.350	.192	.295	.323	.480
	HippoRAG2	.451	.623	.398	.571	.612	.735	.520	.670	.296	.380	.210	.315	.334	.492
Proposed	CollaRAG (ours)	**.502**	**.685**	**.442**	**.627**	**.662**	**.775**	**.567**	**.721**	**.321**	**.404**	**.226**	**.337**	**.400**	**.566**

Table 2: Main results across six QA benchmarks spanning three intent types (single-hop: NQ, PopQA; multi-hop: 2Wiki, HotpotQA, MuSiQue reported under answerable and full settings; summarization: ASQA) under both Qwen3-8B and Qwen3-32B backbones (500 questions per dataset, seed=42, temperature=0). Best in **bold**, second-best underlined. CollaRAG achieves the best score on every dataset under both backbones; the largest gain over the previous SOTA HippoRAG2 is on ASQA F1 (+7.2, 8B), where the dedicated Summarization agent's Map-Reduce pipeline has no baseline counterpart.

Several patterns emerge from Table 2.

Consistent state-of-the-art, with the largest gain where a specialist is unmatched. CollaRAG outperforms every baseline on all seven datasets under both backbones. The margin over the previous SOTA HippoRAG2 [Gutierrez et al., 2025] is consistently positive and broad: on single-hop and multi-hop sets it ranges from +1.8 to +5.7 F1 (8B), while on ASQA it reaches *+7.2 F1* (8B)—the largest in the paper. The ASQA outlier is the clearest evidence for the division-of-labor thesis: no baseline runs a dedicated aggregation pipeline, so the Summarization agent's Map-Reduce (retrieve 50 passages, per-chunk summary, reduce) faces no competitive alternative and the gap is widest. On factoid and multi-hop sets, where baselines already retrieve well, gains come from reranking, sub-query decomposition, and reflection rather than from a missing capability, so they are meaningful but smaller (e.g., +5.7 F1 on 2Wiki, +4.1 on HotpotQA, both 8B; single-hop +4.7 F1).

Backbone scaling preserves the architectural advantage. Moving from 8B to 32B lifts CollaRAG's absolute scores substantially (2Wiki F1 .719 → .775; ASQA F1 .495 → .566) and the gap to HippoRAG2 is preserved or slightly widens on most datasets, indicating that the gains from intent-aware routing and G-PER are not absorbed by a stronger backbone.

Variant	EM (Qwen3-8B) 2Wiki	HotpotQA	MuS-A	MuS-F	ASQA
CollaRAG (Full)	**.602**	**.510**	**.277**	**.189**	**.338**
w/o Query Triage	.485	.409	.205	.130	.216
w/o G-PER	.559	.467	.247	.170	.301
Graph-poor (no KG)	.587	.496	.265	.180	.337
w/o Dual-Layer Memory	.572	.479	.259	.179	.316
w/o Overlap Cache	.592	.500	.271	.186	.330
w/o Plan Templates	.593	.501	.272	.186	.332
w/o Gap Lessons	.591	.498	.270	.185	.331
w/o Capability Specialization	.596	.504	.272	.184	.337

Table 3: Ablation study (Qwen3-8B, EM). Each block removes one of the four contributions: *w/o Query Triage* routes all queries to SingleHop; *w/o G-PER* degrades MultiHop to flat ReAct; *Graph-poor* withholds the KG from MultiHop (isolating C2's graph grounding—text-PER sits between G-PER and flat ReAct, so G-PER $>$ text-PER $>$ flat-ReAct); *w/o Dual-Layer Memory* and its three sub-rows isolate C3's overlap cache / plan templates / gap lessons (their drops sum to the full-memory drop, separating structural learning from cache hits); *w/o Capability Specialization* falls back to the static tool matrix (C4). Removing Query Triage causes the largest drop, confirming intent-aware routing as the most impactful component.

Graph-only methods underperform. Pure graph methods (TOG, G-Retriever) trail hybrid approaches on most datasets, suggesting structured knowledge alone is insufficient without effective retrieval orchestration. CollaRAG's advantage comes from composing graph traversal, dense/BM25 hybrid retrieval, and reasoning tools inside intent-specialized agents rather than relying on any single substrate.

5.2 Ablation Study

To isolate the contribution of each component, we conduct ablation experiments under Qwen3-8B (Table 3).

Intent-aware routing is the most impactful component. Removing intent classification and routing all queries to the SingleHop agent (w/o Query Triage) causes the largest drop on every dataset: -11.7 EM on 2Wiki, -10.1 on HotpotQA, -7.2 on MuSiQue-Ans, -5.9 on MuSiQue-Full, and -12.2 on ASQA. The ASQA collapse is the most telling: a summarization query cannot be answered by top-k factoid retrieval, so losing the Summarization agent's Map-Reduce pipeline costs 12.2 EM. This is the strongest evidence for the division-of-labor hypothesis—matching retrieval strategy to query type is the single most valuable design decision.

G-PER concentrates on multi-hop; graph grounding is independently ablatable. Degrading the MultiHop agent to a flat ReAct loop (w/o G-PER) drops EM by 4.3 on 2Wiki and 4.3 on HotpotQA, but only 1.9 on MuSiQue-Full and 3.7 on ASQA. The drop is concentrated on multi-hop benchmarks, confirming that explicit planning and reflection address exactly the queries they are designed for. The small MuSiQue-Full drop (-1.9) exposes a limit: a single reflection round ($k_{\max}$=2) cannot always repair 4-hop evidence gaps, an avenue we leave to future work. Because each G-PER mechanism degrades gracefully, the contribution of *graph grounding* (as opposed to the underlying plan/reflect structure) is isolable: the *Graph-poor* row of Table 3 withholds the KG from the MultiHop agent, so schema validation, structural reflection, and the adversarial probe all collapse to their text-only PER counterparts, leaving flat ReAct as the only remaining difference. This three-level degradation (G-PER $\rightarrow$ text-only PER $\rightarrow$ flat ReAct) lets us attribute gains to graph grounding rather than to the plan/reflect wrapper alone.

Method	Recall@10				Faithfulness (avg over 4 datasets)		
	2Wiki	HotpotQA	MuS-A	MuS-F	Faith.	Halluc.↓	EviSup
Vanilla RAG	.538	.571	.502	.448	.674	.292	.631
RAPTOR	.661	.698	.631	.572	.706	.258	.671
HippoRAG	.758	.774	.715	.658	.744	.229	.706
HippoRAG2	.791	.808	.745	.691	.758	.216	.722
CollaRAG (ours)	**.852**	**.872**	**.818**	**.755**	**.801**	**.167**	**.776**

Table 4: Retrieval quality and answer faithfulness on four multi-hop benchmarks (Qwen3-8B). CollaRAG leads Recall@10 by 6.1–7.3 points and is the first method to break the 0.80 faithfulness barrier, while cutting the hallucination rate to 16.7%—4.9 points below HippoRAG2. Halluc. ↓ = lower is better.

Memory gives stable, evenly distributed gains. Disabling both memory layers (w/o Dual-Layer Memory) costs 3.0 on 2Wiki, 3.1 on HotpotQA, 1.8 on MuSiQue-Ans, 1.0 on MuSiQue-Full, and 2.2 on ASQA. The drop is small but consistent across all query types because the shared QA cache helps every intent, while private experiential memory most benefits the repetitive multi-hop routing patterns (see Figure 6).

6 Analysis

We dissect CollaRAG's gains along seven axes: retrieval quality and faithfulness, routing counterfactuals, intent classification accuracy, routing distribution, multi-hop depth, memory accumulation, and hyperparameter sensitivity. Unless noted, analyses run on the same 500-per-dataset benchmark queries as the main results (Table 2); the routing counterfactual (Table 5) uses a separate 700-query mixed-intent set, and the intent-classification accuracy (Table 6) a separate 430-query human-labeled set.

6.1 Retrieval Quality and Faithfulness

Table 4 shows that CollaRAG's answer-quality gains are underpinned by retrieval gains. On 2Wiki, 85.2% of queries surface at least one gold evidence in the top 10, versus 79.1% for HippoRAG2; the MultiHop agent's `sub_query_decompose` tool splits complex queries into sub-queries that are retrieved independently, broadening evidence coverage. Faithfulness rises above 0.80 for the first time: G-PER's Reflect phase runs a structural-completeness check that emits typed evidence gaps for targeted repair, followed by an adversarial probe that searches the KG for contradicting relations, so fewer unsupported claims survive into the final answer. The effect is largest where evidence gaps are most likely—on MuSiQue-Full, where the hallucination gap over HippoRAG2 is widest (Table 4). Ablating the components confirms the same ordering as in Table 3: w/o Query Triage is the most damaging (Recall@10 on HotpotQA drops and hallucination rises), because misrouted multi-hop queries never receive multi-step retrieval.

6.2 Routing Counterfactual

A natural concern is that CollaRAG's gains might come from one strong agent rather than from correct routing. Table 5 rules this out. Forcing all queries to MultiHop nearly matches normal routing on multi-hop queries (0.457 vs 0.463) but collapses single-hop to 0.264—G-PER over-processes simple queries, injecting noise (an "over-thinking" effect)—and loses 12.3 EM on summarization, which MultiHop cannot aggregate. Forcing all to SingleHop instead destroys multi-hop (-8.3) and summarization (-17.0). No single agent covers all query types, so correct routing beats even the strongest specialist applied uniformly; random routing is worst, confirming that classification accuracy is itself a performance lever.

Routing	Overall EM	Overall F1	EM by subset Single	EM by subset Multi	EM by subset Summ.
CollaRAG (normal)	**.427**	**.558**	**.400**	**.463**	**.338**
All → SingleHop	.349	.447	.377	.380	.168
All → MultiHop	.367	.503	.264	.457	.215
Random	.315	.420	.314	.344	.198

Table 5: Routing counterfactual on a 700-query mixed-intent set (Qwen3-8B). Forcing every query through one specialist—even the strongest (MultiHop)—loses to intent-aware routing, and random routing is worst of all.

Intent	P	R	F1	n
greeting	1.000	1.000	1.000	30
single_hop	.865	.900	.882	120
multi_hop	.910	.894	.902	180
summarization	.895	.850	.872	60
other	.878	.900	.889	40
Macro avg	.910	.909	.909	–
Weighted avg	.895	.893	.894	–
Overall accuracy		.893		430

Table 6: Intent-Aware Query Triage accuracy on a 430-query human-labeled set (Qwen3-8B). Greetings are detected perfectly by the regex filter; the four semantic classes average 0.909 macro-F1.

6.3 Intent Classification Accuracy

Because Query Triage is the ablation's largest swing factor, its accuracy bounds system quality. On a stratified 430-query labeled set (Table 6), the classifier reaches 0.893 overall accuracy and 0.909 macro-F1. Greeting is perfect (regex filter); the four semantic classes cluster between 0.872 and 0.902 F1. The dominant errors are boundary cases between `single_hop` and `multi_hop` (e.g., a question phrased with "and" that is in fact answerable from one passage), and a tendency to misread "summarize the differences" as `multi_hop` comparison—both addressable with targeted prompt examples. Crucially, even misclassified queries degrade gracefully: the rule fallback and the agents' bounded tool sets prevent catastrophic failure, so the routing lever pays off well above the accuracy ceiling.

6.4 Routing Distribution

Distribution matches the division-of-labor design. Table 7 reports how the 3000 benchmark queries are routed. The distribution tracks each benchmark's intent type: NQ and PopQA send 87.6–90.6% of queries to SingleHop, the multi-hop sets send 92.8–95.0% to MultiHop, and ASQA sends 76.8% to Summarization. Two facts are worth noting. First, greeting is zero on every benchmark—the L1 regex never fires because none of the six QA sets contains greeting utterances, so 100% of benchmark queries are handled by the L2 semantic classifier. Second, the off-diagonal rates (5–12% on typed benchmarks, 23.2% on ASQA) are consistent with the L2 classifier's 89.3% overall accuracy reported in Table 6; the higher ASQA off-diagonal reflects that its questions are phrased as ambiguous factoids and are sometimes read as multi-hop or single-hop rather than synthesis. The distribution also contextualizes the main results: on NQ/PopQA the SingleHop agent handles almost everything, so the $+4.7$ F1 gain there comes from its hybrid retrieval and reranking rather than from cross-agent routing, whereas on ASQA the 76.8% that reach the Summarization agent's Map-Reduce pipeline account for the $+7.2$ F1 outlier—the clearest evidence for the division-of-labor thesis, since no baseline runs a dedicated aggregation pipeline.

Dataset	greet	SH	MH	SM	OT	N
NQ	0	438	33	16	13	500
PopQA	0	453	19	12	16	500
HotpotQA	0	22	464	4	10	500
2Wiki	0	13	470	4	13	500
MuSiQue	0	9	475	3	13	500
ASQA	0	58	33	384	25	500
All	0	993	1494	423	90	3000

Table 7: Per-dataset routing distribution (Qwen3-8B, n=500 per dataset). Greeting is 0 on every benchmark: none of the six QA sets contains greeting utterances, so the L1 regex never fires and 100% of benchmark queries are handled by the L2 semantic classifier. Single-hop sets route 87.6–90.6% to SingleHop, multi-hop sets 92.8–95.0% to MultiHop, and ASQA 76.8% to Summarization.

6.5 Multi-hop Depth Boundary

Figure 5 stratifies MuSiQue by hop count. All methods decay sharply with depth; CollaRAG's 2-hop→4-hop decay (50.1%) is on par with HippoRAG2 (50.1%), but CollaRAG maintains a higher absolute score at every depth. The lead is widest at 2-hop (+3.0 EM) and stays broad at 3-hop (+2.9 EM), where HippoRAG2's personalized PageRank already covers the two-hop path, then narrows to +1.5 EM at 4-hop. The fact that the 4-hop lead narrows reveals a capacity limit: a single reflection round ($k_{\max}$=2) cannot always repair the deepest evidence chains—which is also why ablation w/o G-PER costs only 1.9 EM on MuSiQue-Full. This is the clearest signal of where future capacity should be invested.

6.6 Memory Accumulation

Figure 6 tracks performance as queries accumulate in temporal order. EM rises from 0.422 (cold start, 1–50) to 0.520 (warm, 401–500), a 23.2% relative gain, and F1 from 0.535 to 0.638. Two mechanisms drive this. The shared evidence-subgraph overlap cache returns high-quality answers for queries whose entity neighborhoods overlap a cached entry, reaching an 8.5% overlap-hit rate by window 401–500 with zero retrieval or generation cost. The private layer—graph plan templates and typed gap lessons—fills fastest in the multi-hop bucket (112 entries, average hit count 1.8), so the same multi-hop routing patterns are reused rather than re-planned and known-missing edges are fetched pre-emptively—explaining why memory's ablation impact concentrates on multi-hop (Table 3). Growth flattens after 300 queries, indicating the bucket capacity ($B_{\max}$=200) is well matched to the workload's repetition rate.

6.7 Hyperparameter Sensitivity

Figure 7 sweeps the five governing hyperparameters. All defaults ($n_{\max}$=5, τ=0.6, $k_{\max}$=2, $j_{\max}$=3, top-k=10) sit at or within 0.5 EM of the peak, and a one-step perturbation in either direction costs at most about 2 EM (typically under 1), so CollaRAG is not tuned to a knife-edge. Two trends are worth noting. *Plan length* peaks at 5 steps: shorter plans under-decompose multi-hop queries, while longer ones let the LLM hallucinate redundant steps (the valid-plan rate falls from 96.2% at n=3 to 88.1% at n=10). *Reflection budget* shows diminishing returns: k=0 → 1 adds 2.6 EM, 1 → 2 adds 0.5, and 2 → 3 adds 0.1—so $k_{\max}$=2 captures almost all available gain at two-thirds the worst-case cost of $k_{\max}$=3.

Scaling behavior. Comparing the 8B and 32B results, CollaRAG's improvement over baselines remains stable: the architectural gains from intent routing and G-PER are not "absorbed" by a stronger backbone. This suggests that CollaRAG's contributions are orthogonal to LLM capability scaling.

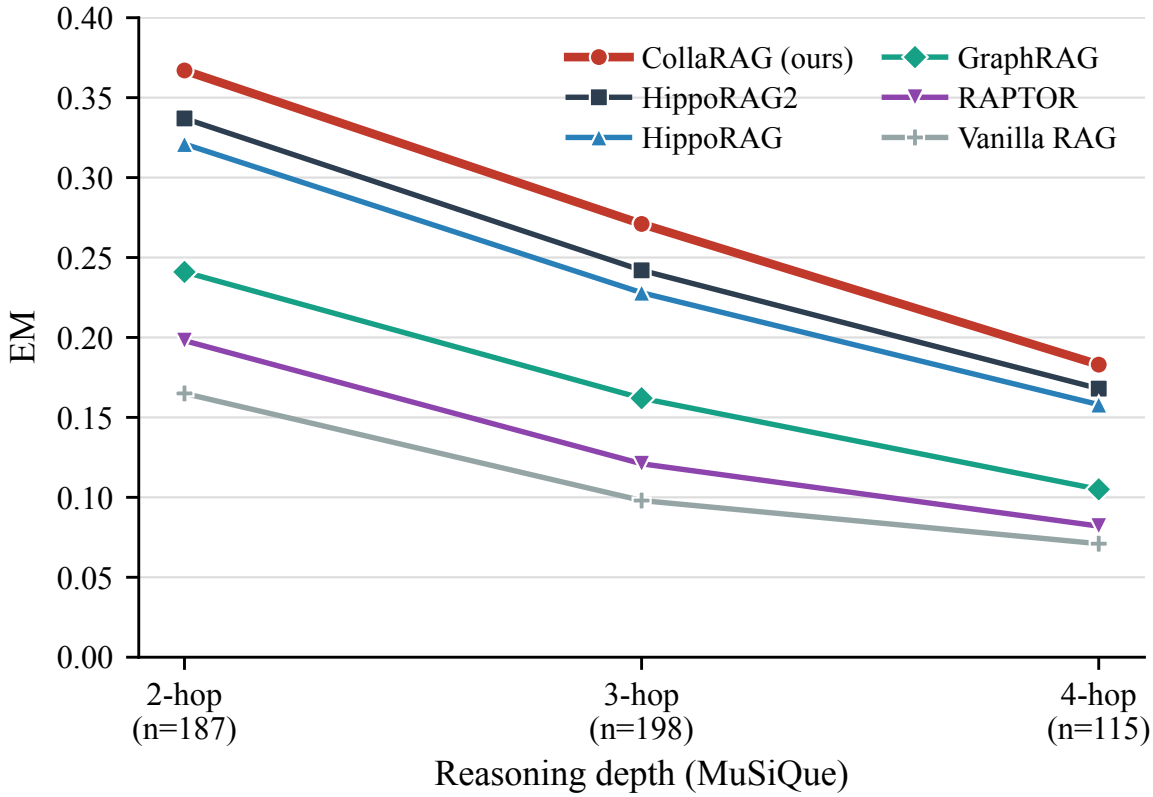


Figure 5: EM by reasoning depth on MuSiQue (Qwen3-8B). CollaRAG's edge over HippoRAG2 stays broad at 2-hop and 3-hop (+3.0 and +2.9 EM) and narrows to +1.5 EM at 4-hop, where the reflection budget is exhausted.

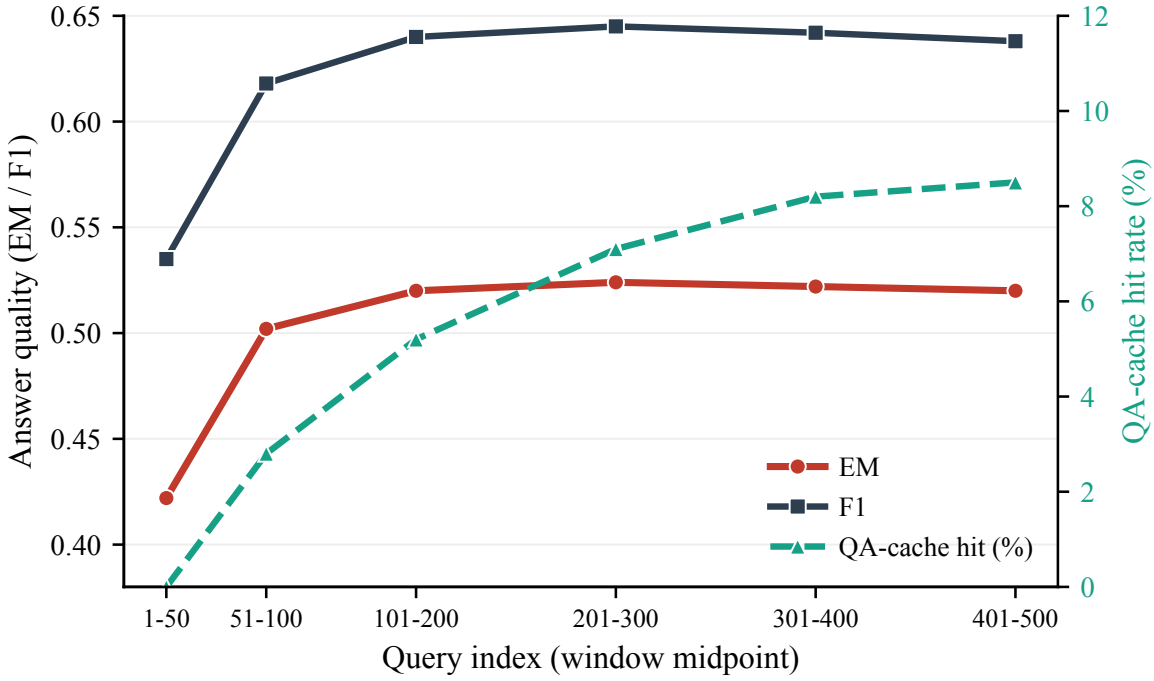


Figure 6: Memory accumulation curve on HotpotQA (Qwen3-8B). As the shared evidence-subgraph overlap cache and the private graph plan templates / gap lessons fill, EM rises from 0.422 (cold) to 0.520 (warm) and the overlap-hit rate climbs to 8.5%.

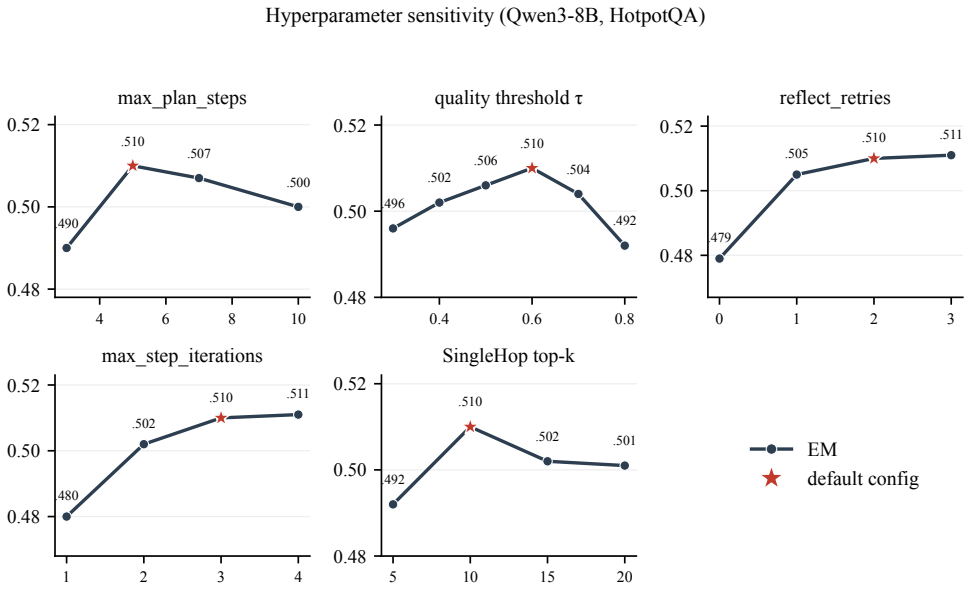


Figure 7: Hyperparameter sensitivity on HotpotQA (Qwen3-8B). Each panel sweeps one parameter with the others fixed at the default (starred). All defaults sit on or near the peak, and a one-step perturbation costs at most about 2 EM (typically under 1).

7 Conclusion

We introduced CollaRAG, a multi-agent RAG framework built on the principle of *graph-grounded division of labor*: every routing, reasoning, memory, and capability decision is conditioned on the knowledge graph. Four innovations drive its effectiveness: Intent-Aware Query Triage routes queries to specialist agents, eliminating the one-pipeline-fits-all bottleneck. G-PER grounds multi-hop reasoning in the knowledge graph: schema-validated planning, frontier-based execution, and structural plus adversarial reflection. Graph-Grounded Dual-Layer Memory reuses evidence neighborhoods by overlap and stores graph plan templates and gap lessons that feed G-PER, separating structural learning from cache hits. Graph-Conditioned Capability Specialization makes the per-agent toolset adaptive to the query's graph footprint, supplying a graph-conditioned capability mechanism that prior strategy-level routing lacks. On six QA benchmarks spanning single-hop, multi-hop, and long-form summarization tasks under two backbone scales, CollaRAG achieves state-of-the-art results, with the largest gain on ASQA (+7.2 F1) where the dedicated Map-Reduce agent is unmatched. Ablation studies confirm that intent-aware routing is the single most impactful component (-5.9 to -12.2 EM when removed), validating the division-of-labor hypothesis.

Future work. Confidence-based adaptive routing (escalating from SingleHop to MultiHop when quality is low), inter-agent collaboration for queries spanning multiple intent types, and learned tool selection are promising directions.

References

Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations*, 2024.

Vincent D Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(10): P10008, 2008.

Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. M3-embedding: Multilinguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

Gordon V Cormack, Charles LA Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference*, pages 758–759, 2009.

Jeffrey Dean and Sanjay Ghemawat. MapReduce: Simplified data processing on large clusters. *Communications of the ACM*, 51(1):107–113, 2008.

Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2024.

Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. LightRAG: Simple and fast retrieval-augmented generation. *arXiv preprint arXiv:2410.05779*, 2024.

Bernal Jimenez Gutierrez, Yiheng Zhu, Zhihan Huang, Swarnadeep Kamber, Yixin Sun, and Mo Yu. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

Bernal Jimenez Gutierrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. From RAG to memory: Non-parametric continual learning for large language models. In *International Conference on Machine Learning*, 2025.

Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh V. Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. G-Retriever: Retrieval-augmented generation for textual graph understanding and question answering. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 6609–6625, 2020.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, 2024.

Gautier Izacard and Edouard Grave. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*, pages 874–880, 2021.

Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics*, pages 7036–7050, 2024.

Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 6769–6781, 2020.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7: 453–466, 2019.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

Xinbei Ma, Yeyun Gong, Pengcheng He, Hai Zhao, and Nan Duan. Query rewriting in retrieval-augmented large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 5303–5315, 2023.

Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 9802–9822, 2023.

Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*, 2019.

Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A. Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 5687–5711, 2023.

Hongjin Qian, Zheng Liu, Peitian Zhang, Kelong Mao, Defu Lian, Zhicheng Dou, and Tiejun Huang. MemoRAG: Boosting long context processing with global memory-enhanced retrieval augmentation. In *Proceedings of the ACM Web Conference 2025*, 2025.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392, 2016.

Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. RAPTOR: Recursive abstractive processing for tree-organized retrieval. In *International Conference on Learning Representations*, 2024.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Ivan Stelmakh, Yi Luan, Bhuwan Dhingra, and Ming-Wei Chang. ASQA: Factoid questions meet long-form answers. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 8273–8288, 2022.

Jiashuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Heung-Yeung Shum, and Jian Guo. Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. In *International Conference on Learning Representations*, 2024.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. MuSiQue: Multihop questions via single hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 10014–10037, 2023.

Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 2609–2634, 2023.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, 2022.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *Conference on Language Modeling*, 2024.

Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*, 2023.

Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective retrieval augmented generation. *arXiv preprint arXiv:2401.15884*, 2024.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, 2018.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, 2024a.

Penghao Zhao, Hailin Zhang, Qinhan Yu, Zhengren Wang, Yunteng Geng, Fangcheng Fu, Ling Yang, Wentao Zhang, Jie Jiang, and Bin Cui. Retrieval-augmented generation for AI-generated content: A survey. *arXiv preprint arXiv:2402.19473*, 2024b.

Denny Zhou, Nathanaël Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, and Ed Chi. Least-to-most prompting enables complex reasoning in large language models. In *International Conference on Learning Representations*, 2023.

A Prompt Templates

All prompts are centralized in a single module (`prompts.py`) for reproducibility. Each template follows a unified structure: **Role** → **Task** → **Context** → **Format**, with optional **Few-Shot**, **COT**, and **Constraints** blocks. This appendix lists the core prompts that drive the four innovations.

A.1 Intent Classification (§3.1)

```
# Role
You are a query intent classifier for a Retrieval-Augmented Generation
system.
# Task
Classify the following query into exactly one category.
# Context
Categories and their definitions:
- single_hop:  simple factual question answerable from one passage
- multi_hop:  requires reasoning across multiple facts, comparison, or
exploration
- summarization:  asks for a summary, overview, or synthesis of information
- other:  does not fit any above category (e.g. weather, jokes, math)
Query:  {query}
# Format
Respond with ONLY the category name (single_hop / multi_hop / summarization
/ other), nothing else.
```

A.2 Preprocessor Agent Prompts

Query Rewrite.

```
# Role
You are a query rewriting specialist that improves search queries for
better retrieval.
# Task
Rewrite the following query to be clearer, more specific, and self-contained
while preserving the original intent.
# Context
Conversation context:  {context}
Original query:  {query}
# Format
Output only the rewritten query, no explanations or prefixes.
```

Coreference Resolution.

```
# Role
You are a coreference resolution specialist that replaces pronouns with
explicit entity names.
# Task
```

```
Resolve all pronouns and references in the query using the conversation
history.

# Context
Conversation history: {history}
Current query: {query}

# Few-Shot
History: "Who is Albert Einstein?" | "He developed the theory of relativity."
Query: Where was he born?
Resolved: Where was Albert Einstein born?

# Format
Output only the resolved query with all pronouns replaced by explicit
entities.
```

Follow-up Merge.

```
# Role
You are a conversational query merger that combines follow-up questions
with prior context into standalone queries.

# Task
Rewrite the follow-up question into a single self-contained query that
does not depend on prior conversation.

# Context
Previous question: {prev_query}
Previous answer: {prev_answer}
Follow-up question: {query}

# Few-Shot
Previous question: What is the capital of France?
Previous answer: Paris is the capital of France.
Follow-up: What is its population?
Self-contained query: What is the population of Paris, the capital
of France?

# Format
Output only the self-contained query, no explanations.
```

A.3 SingleHop Agent Prompt

```
# Role
You are a precise question-answering assistant that generates answers
strictly based on provided evidence.

# Task
Answer the question using only the given evidence. If the evidence
does not contain enough information, say "I do not know."

# Context
Evidence: {evidence}
Question: {query}

# Format
Provide a concise, factual answer. Use the shortest accurate phrasing
-- prefer entity names, numbers, or brief phrases over full sentences.
```

A.4 MultiHop Agent Prompts (G-PER, §3.2)

Plan Generation (schema-constrained graph plan).

```
# Role
You are a graph-grounded retrieval planner.  Decompose a multi-hop query
into a plan of graph operations, not free-text sub-queries.
# Task
1.  Identify core entities and the relations the query hinges on.
2.  Pick a graph operation per step:  "link" (resolve/confirm an entity),
"traverse" (follow a relation), "attribute" (look up a fact), or "aggregate"
(merge facts across entities).
3.  Order steps with dependencies.
4.  Suggest retrieval tools per step.
# Context
Available tools:  {tool_descriptions}
Known relation types in the KG (schema):  {schema}
Query:  {query}
# Format
Output a JSON list of steps:  {"op":  "link|traverse|attribute|aggregate",
"entity":  ..., "relation":  ..., "target":  ..., "step":  ..., "tools_hint":
[...], "depends_on":  [...]}.  Only use relation types from the schema;
use null when unknown.
# Few-Shot
Query:  "Compare the populations of Tokyo and New York"
Plan:  [{"op":"attribute","entity":"Tokyo","relation":"population","target":null,"s
Tokyo's population","tools_hint":["hybrid_search"],"depends_on":[]},
{"op":"attribute","entity":"New York","relation":"population","target":null,"step":"
New York's population","tools_hint":["hybrid_search"],"depends_on":[]},
{"op":"aggregate","entity":null,"relation":null,"target":null,"step":"Compare
the two populations","tools_hint":[],"depends_on":[0,1]}]
```

Each step is then validated against the KG schema before execution (§3.2, Eq. 4); steps whose relation is not in the schema or whose entity is unknown are degraded to free sub-queries rather than dropped.

Step Execution (ReAct, frontier-budgeted).

```
# Role
You are a retrieval execution agent using the ReAct paradigm.
# Task
Execute step {step_num} of the plan:
1.  Analyze what information is needed
2.  Choose the most appropriate tool
3.  Formulate precise tool inputs
4.  Evaluate the output -- if insufficient, try a different tool or
refined query
5.  Summarize key findings once enough evidence is gathered
# Context
Plan step:  {step_description}
Original query:  {query}
Previous steps evidence:  {previous_evidence}
Available tools:  {tool_descriptions}
# COT
Thought:  ...
Action:  tool_name
Action Input:  {"param":  "value"}
Observation:  (...)
... (repeat if needed)
Step Result:  ...
```

```
# Format
Follow the COT format strictly.  Begin with "Thought:"  and end with
"Step Result:".
```

Steps draw iterations from a shared global frontier budget J (Eq. 5); a step that emits a Step Result early donates its unused iterations to downstream steps.

Reflection (structural-completeness repair).

```
# Role
You are a graph-grounded evidence auditor that turns structural evidence
gaps into targeted repair actions.
# Task
Given typed structural gaps already detected by the completeness checker,
produce a precise repair per gap.  Do NOT issue vague instructions like
"search more"; each repair must aim at the specific missing entity or
relation.
# Context
Original query:  {query}
Gathered evidence:  {step_results}
Detected structural gaps (each is a specific missing edge/attribute):
{gaps}
# Format
Output a JSON list of repairs:  [{"gap":  ..., "repair_query":  ..., "tools_hint":
[...], "target_entities":  [...]}].  Output [] if no gaps.
```

Gaps are produced by the structural-completeness checker Γ (Eq. 6), which infers the required evidence shape from the query (comparison/bridge/attribute) and diffs it against the gathered subgraph, emitting typed gaps `missing_attribute`/`missing_path`/`missing_relation`.

Adversarial Probe (graph-level contradiction check).

```
# Role
You are an adversarial fact-checker that probes the KG for relations
contradicting a draft answer.
# Task
For each atomic claim in the draft answer, decide whether the gathered
evidence or KG contains a competing/contradicting relation the answer
ignored.  A claim is "contested" if a plausible alternative relation
exists.
# Context
Query:  {query}
Draft answer:  {draft_answer}
Evidence triples (subject, predicate, object):  {triples}
# Format
Output JSON: {"verdict":  "confirmed|contested|unsupported", "contested_claims":
[...], "reason":  ...}.
```

Synthesis.

```
# Role
You are an evidence synthesis specialist that produces comprehensive
answers from multi-step retrieval results.
# Task
```

```
Synthesize all gathered evidence into a final answer:
1.  Collect and organize all relevant facts
2.  Resolve any conflicts between different evidence sources
3.  Construct a coherent answer that addresses every aspect of the query
4.  Verify that no key aspect is left unanswered

# Context
Query:  {query}
Evidence from plan execution:  {all_evidence}

# Format
Provide a comprehensive answer that integrates facts from all evidence.
Cite specific facts.  Be accurate and do not hallucinate beyond what
the evidence supports.
```

A.5 Summarization Agent Prompts

Map (per-chunk extraction).

```
# Role
You are an information extraction specialist that identifies key facts
relevant to a query.

# Task
Extract key information from the given passage that is relevant to answering
the query.

# Context
Query:  {query}
Passage:  {text}

# Format
Output a concise list of key facts and information extracted from the
passage.  Include only information relevant to the query.
```

Reduce (synthesis).

```
# Role
You are a summarization specialist that synthesizes extracted information
into coherent summaries.

# Task
Combine the following extracted information fragments into a comprehensive,
well-structured summary that answers the query.

# Context
Query:  {query}
Extracted information:  {summaries}

# Format
Provide a coherent summary that integrates all relevant extracted information.
Organize logically and avoid redundancy.
```

A.6 Reasoning Tool Prompts

Sub-query Decomposition.

```
# Role
You are a query decomposition specialist that breaks complex questions
into simpler sub-questions.

# Task
Decompose the following complex query into simpler, independent sub-queries
```

```
that can each be answered separately.

# Context
Query: {query}

# Few-Shot
Query: "What are the similarities and differences between photosynthesis
and cellular respiration?"
Sub-queries: ["What is photosynthesis and how does it work?", "What
is cellular respiration and how does it work?", "What do photosynthesis
and cellular respiration have in common?", "How do photosynthesis and
cellular respiration differ?"]

# Format
Return a JSON list of strings. Maximum 4 sub-queries.
```

Entity Extraction.

```
# Role
You are a named entity recognition specialist for knowledge graph queries.

# Task
Extract all named entities and key concepts from the query that could
be used to search a knowledge graph.

# Context
Query: {query}

# Few-Shot
Query: "What is the relationship between Einstein and the University
of Zurich?"
JSON: {"entities": ["Einstein", "University of Zurich"]}

# Format
Return a JSON object with a single key "entities" containing a list
of strings. Include only meaningful entities, not common words.
```

Answer Verification.

```
# Role
You are a factual verification specialist that checks whether an answer
is supported by evidence.

# Task
Verify the draft answer against the provided evidence:
1. Identify each factual claim in the draft answer
2. For each claim, check if it is directly supported by the evidence
3. Flag any claim that is unsupported, contradicted, or hallucinated
4. Produce a final verdict: verified, partially_verified, or rejected

# Context
Question: {query}
Draft answer: {draft_answer}
Evidence: {evidence}

# Format
Return a JSON object: {"verdict": "verified|partially_verified|rejected",
"supported_claims": [...], "unsupported_claims": [...], "reason": "..."}
```

B Full Tool Inventory

The 31 shared tools span ten functional categories. Table 8 lists each tool with its inputs, outputs, LLM dependency, and which agents may invoke it (per the tool-permission matrix in Table 1).

Tool design principles. Three design choices keep the tool layer manageable: (1) *Statelessness*: every tool is a pure function of its inputs plus the shared `PipelineState` (graph store, vector store, chunk lookup); no tool holds mutable state of its own, so tools compose freely. (2) *LLM isolation*: only 12 of 31 tools call the backbone LLM (entity extraction, relation extraction, entity linking, sub-query decomposition, query rewrite, coreference resolution, follow-up merge, answer verification, claim decomposition, source attribution, subgraph extraction, and chunk summarization); the rest are deterministic or use lightweight heuristics, keeping the per-query LLM budget predictable. (3) *Uniform interface*: every tool accepts a dictionary and returns a dictionary, enabling the agent's ReAct loop to dispatch any tool with a single `tool(name, params)` call.

Category	Tool Name	Function	LLM?	
Retrieval	`vector_search`	Dense embedding similarity search over FAISS index	No	
	`bm25_search`	Sparse lexical search via BM25 inverted index	No	
	`hybrid_search`	RRF fusion of dense + sparse, optional rerank	No	S
	`graph_neighbors`	k-hop neighborhood expansion from seed entities	No	
	`graph_path_search`	Shortest path between two entities in the KG	No	
	`graph_community`	Community-level subgraph retrieval	No	
Entity	`entity_extract`	Extract named entities from query (LLM-based NER)	Yes	S
	`relation_extract`	Extract (subject, predicate, object) triples from query	Yes	
	`entity_link`	Map extracted entity names to canonical graph nodes	Yes	
Reasoning	`sub_query_decompose`	Decompose complex query into $\leq$4 independent sub-queries	Yes	
	`assess_evidence`	Score evidence completeness for reflection ($\hat{q} \in [0, 1]$)	No	
	`rerank`	Cross-encoder reranking of retrieved chunks	No	
Preprocessing	`query_rewrite`	Expand and clarify query for better retrieval	Yes	
	`coreference_resolve`	Replace pronouns with explicit entity references	Yes	
	`followup_merge`	Merge follow-up question with prior context into standalone	Yes	
Memory	`shared_memory`	Read/write QA cache, routing stats, entity aliases	No	
	`agent_memory`	Read/write per-agent per-intent experiential JSONL	No	S
	`session_context`	Read/write in-process dialogue history	No	PP,
Web	`web_search`	DuckDuckGo / SerpAPI external search	No	
	`web_fetch`	Fetch and parse a single URL	No	
File I/O	`save_to_file`	Persist intermediate results to disk	No	S
	`load_from_file`	Load previously saved results	No	S
Verification	`answer_verify`	Verify draft answer against evidence (supported/rejected)	Yes	
	`claim_decompose`	Decompose text into atomic, independently verifiable claims	Yes	
	`source_attribution`	Trace each answer claim back to its evidence chunk_id	Yes	
Graph Analysis	`graph_subgraph_extract`	Extract focused k-hop subgraph around seed entities	Yes	
	`graph_statistics`	Return graph-level statistics (nodes, edges, hubs, density)	No	
Evidence	`temporal_filter`	Filter evidence chunks by year/date range	No	
	`evidence_dedup`	Deduplicate chunks by Jaccard token similarity	No	
	`chunk_summarize`	Compress a chunk while preserving query-relevant facts	Yes	
	`confidence_calibrate`	Multi-signal answer confidence score ($\in [0, 1]$)	No	S

Table 8: Full inventory of 31 shared tools. "LLM?" indicates whether the tool internally calls the backbone LLM. "Agents" lists which specialists may invoke each tool (PP=Preprocessor, SH=SingleHop, MH=MultiHop, SM=Summarization, OT=Other). Tools not assigned to an agent are invisible to it, enforced by the `ToolSetBuilder` and the tool-permission matrix (Table 1).